

KNOWLEDGE CORPUS • ARCHITECTURE SPECIFICATION

AWS Cloud Compute & Container Infrastructure

A Structured Technical Reference on EC2, Elastic Container Service (ECS), Elastic Kubernetes Service (EKS), AWS Fargate, and IAM Security

Doc ID: GL-CORPUS-04	Domain: Cloud Infrastructure	Topic: AWS Compute & Containers	Target: Graph RAG System
----------------------	------------------------------	---------------------------------	--------------------------

1. Prerequisites & Everyday Cloud Analogies

Before diving into cloud compute services, let us understand how modern cloud infrastructure works using an intuitive real-world analogy.

Imagine you run a shipping company. In the traditional IT model, you had to buy physical warehouses (servers) and build private roads (cables). In the cloud computing model provided by Amazon Web Services (AWS), you rent virtual space and compute power by the second:

- **Virtual Machines (Amazon EC2):** Renting an empty office building. You have full control over the doors, windows, and furniture (operating system and software updates), but you are also responsible for maintaining and cleaning it.
- **Containers (Docker Images):** Standardized shipping boxes. Everything inside the box (code, libraries, configuration) is packed together so it can be shipped anywhere instantly without worrying about the underlying truck or ship.
- **Managed Container Platforms (Amazon ECS & Amazon EKS):** An automated dispatcher and harbor crane system that decides which shipping boxes go onto which ships and handles loading, unloading, and scaling automatically.
- **Serverless Compute (AWS Fargate):** A courier delivery service. You hand over the package (your container) and specify where it needs to go. AWS takes care of the trucks, drivers, and roads without you ever seeing or managing a server.

2. Core AWS Compute & Container Concepts

This section introduces the foundational building blocks of the AWS compute ecosystem.

Concept: Amazon EC2 (Elastic Compute Cloud)

- Beginner-Friendly Definition:** Amazon EC2 provides on-demand, resizable virtual servers (called instances) in the cloud. You can choose how much CPU, memory, and storage your server needs and run any software you want on it.
- Why It Exists:** To eliminate the need to purchase, rack, power, and maintain expensive physical hardware in on-premises datacenters.
- How It Works:** AWS provisions virtual machines on top of its physical server fleet using the AWS Nitro hypervisor. Users can launch, stop, resize, and terminate instances in seconds via the AWS Management Console or AWS CLI.
- Real-World Cloud Example:** Launching a fleet of EC2 t4g.xlarge instances running Amazon Linux to act as worker nodes for your container cluster.
- Related Concepts:** Virtual Machine, AWS Nitro, Auto Scaling Group, Amazon Machine Image (AMI), Elastic Block Store (EBS).

Concept: Amazon ECS (Elastic Container Service)

Beginner-Friendly Definition: Amazon ECS is AWS's native, highly scalable container management service designed to run, stop, and manage Docker containers on a cluster without requiring complex Kubernetes setup.

Why It Exists: To provide a simple, powerful, and deeply integrated container orchestrator that works natively with all AWS services out of the box.

How It Works: You define your application in a *Task Definition* (specifying container images, CPU, and memory requirements). ECS launches these tasks across your chosen compute layer (EC2 instances or AWS Fargate) and maintains the desired number of running copies.

Real-World Cloud Example: Deploying a backend Node.js API across 10 ECS Tasks connected to an AWS Application Load Balancer and an Amazon Aurora database.

Related Concepts: Task Definition, ECS Task, ECS Service, AWS Fargate, Amazon ECR.

Concept: Amazon EKS (Elastic Kubernetes Service)

Beginner-Friendly Definition: Amazon EKS is a fully managed service that makes it easy to run Kubernetes on AWS without needing to install, operate, or maintain your own Kubernetes control plane.

Why It Exists: Running a production Kubernetes control plane manually across multiple datacenters requires complex etcd backups, certificate rotations, and high-availability engineering. EKS automates these operational burdens.

How It Works: AWS provisions and manages a highly available, multi-Availability Zone Kubernetes control plane (kube-apiserver and etcd). Users connect their worker nodes (EC2 instances or Fargate profiles) to this control plane and deploy standard Kubernetes manifests using `kubectl`.

Real-World Cloud Example: Migrating an enterprise microservices platform from an on-premises datacenter to Amazon EKS for automated scaling and disaster recovery.

Related Concepts: Kubernetes, Control Plane, Worker Node, Kubelet, Managed Node Groups, AWS IAM.

Concept: AWS Fargate (Serverless Container Compute)

Beginner-Friendly Definition: AWS Fargate is a serverless compute engine for containers that works with both Amazon ECS and Amazon EKS. With Fargate, you do not have to provision, configure, patch, or scale virtual machine servers.

Why It Exists: Managing virtual machine clusters involves operating system security patching, capacity planning, and server maintenance. Fargate lets developers focus entirely on writing application code.

How It Works: You package your application into a container and specify the required CPU and memory (e.g., 0.5 vCPU and 1 GB RAM). AWS automatically provisions an isolated, dedicated compute environment on demand and bills you only for the exact seconds the container runs.

Real-World Cloud Example: Running batch data processing jobs or unpredictable webhook microservices on AWS Fargate without keeping idle EC2 servers running 24/7.

Related Concepts: Serverless, Amazon ECS, Amazon EKS, Pay-as-you-go, MicroVM Isolation.

3. AWS Container Compute Comparison

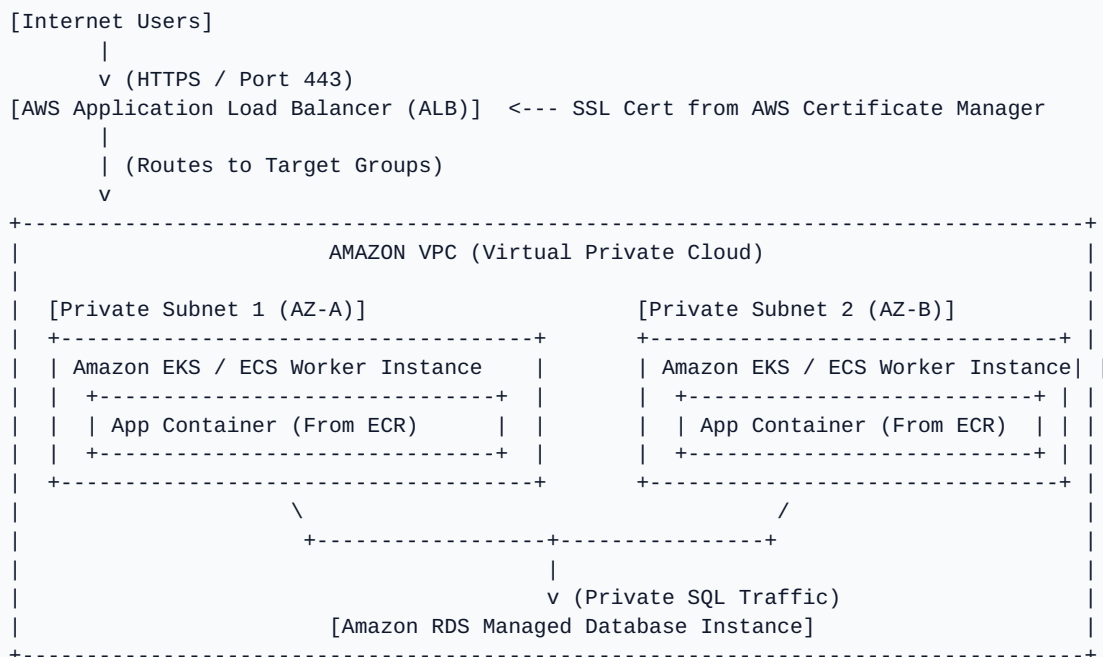
Choosing the right compute and orchestration model depends on operational complexity, governance requirements, and application architecture. The table below compares the primary container options on AWS:

Feature / Dimension	Amazon ECS + EC2	Amazon EKS + EC2	AWS Fargate (Serverless)
Orchestrator	AWS Native ECS Engine	Standard Kubernetes (CNCF)	Works with ECS or EKS
Server Management	You manage and patch EC2 VMs	You manage EC2 worker nodes	Zero server management (Managed by AWS)
Learning Curve	Low — Simple AWS concepts	High — Comprehensive Kubernetes ecosystem	Lowest — Define CPU/RAM and run
AWS Integration	Deep native integration	Integrated via AWS CNI, CSI, and ALB plugins	Native AWS integration
Pricing Model	Pay for underlying EC2 VMs	EKS cluster fee (\$0.10/hr) + EC2 VMs	Pay per vCPU/RAM per second consumed
Best For	AWS-centric architectures, simpler workflows	Large multi-cloud teams, complex microservices	Variable workloads, microservices, batch jobs

4. Application Deployment Architecture on AWS

In a typical cloud microservices deployment, multiple AWS services work together to deliver scalable, resilient, and secure application traffic:

- Image Storage (Amazon ECR):** Developers build Docker images and push them to **Amazon Elastic Container Registry (Amazon ECR)**, a secure, fully managed container image registry.
- Traffic Ingress (AWS Application Load Balancer):** Incoming client web requests hit an **Application Load Balancer (ALB)**. The ALB terminates SSL/TLS certificates (managed by AWS Certificate Manager) and distributes HTTP/HTTPS traffic across healthy container instances.
- Workload Execution (EKS / ECS):** The orchestrator (EKS or ECS) pulls the container image from ECR and schedules tasks across EC2 worker instances or AWS Fargate compute.
- Network Isolation (AWS VPC & Security Groups):** Containers run inside private subnets within an **Amazon Virtual Private Cloud (VPC)**. Virtual firewalls called **Security Groups** control inbound and outbound network traffic to ensure only authorized connections are permitted.
- Stateful Storage (Amazon RDS & DynamoDB):** Application containers store relational data in **Amazon Relational Database Service (Amazon RDS)** or document data in **Amazon DynamoDB**, keeping the container tier completely stateless and easy to scale.



5. Security & Access Control with AWS IAM

Security in AWS follows the principle of least privilege. In traditional server environments, applications were often granted broad server-wide credentials. Modern container architectures use granular, role-based access control.

5.1 IAM Roles for Service Accounts (IRSA) in Amazon EKS

In an EKS cluster, multiple different microservices may run on the same EC2 worker node. If the EC2 host had broad permissions (e.g., access to an Amazon S3 bucket), every container on that host could access that bucket.

IAM Roles for Service Accounts (IRSA) solves this problem by binding AWS Identity and Access Management (IAM) roles directly to specific Kubernetes *ServiceAccounts*:

- An OpenID Connect (OIDC) identity provider is established between your EKS cluster and AWS IAM.
- You attach an IAM Role (e.g., `S3ReadOnlyRole`) to a Kubernetes *ServiceAccount* (e.g., `billing-app-sa`).
- When the `billing-app` Pod starts, the AWS SDK inside the container automatically requests temporary, rotating security credentials specifically for that IAM role.
- Other Pods running on the exact same EC2 instance have zero access to the S3 bucket unless explicitly granted their own IAM roles.

5.2 ECS Task Execution Roles vs. Task Roles

In Amazon ECS, permissions are divided into two distinct IAM roles:

- **Task Execution Role:** Used by the ECS agent itself before the application starts. This role grants permission to pull container images from private Amazon ECR repositories and send container logs to Amazon CloudWatch.
- **Task Role:** Used by the actual application code inside the container during runtime. This role defines which AWS resources the application can access (e.g., reading from DynamoDB or publishing messages to Amazon SQS).

6. Practical Operational Troubleshooting

When deploying containerized workloads on AWS, common operational issues generally involve permissions, capacity limits, or load balancer health checks.

Failure Symptom	Root Cause	Diagnostic Action	Remediation Action
AccessDenied / 403 Forbidden	The application container lacks IAM permissions to access an AWS resource (e.g., S3 or DynamoDB).	Check AWS CloudTrail logs and verify the Pod's ServiceAccount annotation or ECS Task Role.	Attach the required IAM policy actions (e.g., <code>s3:GetObject</code>) to the assigned IAM role.
ALB 502 Bad Gateway / Unhealthy Targets	Containers failed ALB health checks because the application did not respond on the health check path.	Inspect ALB Target Group health check status and review container application startup logs.	Ensure application listens on the correct port (e.g., <code>0.0.0.0:8080</code>) and responds with HTTP 200 on <code>/healthz</code> .
CannotPullContainerError (ECR)	ECS/EKS node cannot reach or authenticate with the Amazon ECR repository.	Inspect ECS task stopped reason or run <code>kubectl describe pod <pod></code> .	Verify VPC NAT Gateway routing, check ECR IAM pull permissions, or configure AWS PrivateLink for ECR.
InsufficientInstanceCapacity	AWS datacenter Availability Zone is temporarily out of the requested EC2 instance type.	Check EC2 Auto Scaling Group activity logs.	Update Auto Scaling configuration to use multiple instance types (e.g., mixing <code>m5.large</code> , <code>m5a.large</code> , and <code>m6i.large</code>).

7. Explicit Architectural Relationships & Dependencies

This section explicitly documents the structural dependencies between AWS compute, container services, and supporting cloud components for Graph RAG processing:

- **Amazon EKS manages Kubernetes Control Planes:** Amazon EKS provisions, updates, and scales managed Kubernetes control plane components across multiple AWS Availability Zones.
- **Amazon EKS launches worker nodes on Amazon EC2:** Amazon EKS uses EC2 instances configured in Managed Node Groups to execute containerized Pods.
- **Amazon ECS executes Task Definitions:** Amazon ECS reads JSON task definitions to configure container images, port mappings, and resource allocations.
- **AWS Fargate provides serverless compute for ECS and EKS:** AWS Fargate runs container workloads on demand without requiring virtual machine provisioning or server management.
- **Amazon ECR stores container images:** Amazon Elastic Container Registry provides secure, managed image hosting consumed by Amazon ECS, Amazon EKS, and AWS Fargate.
- **AWS Application Load Balancer routes traffic to containers:** An ALB distributes incoming user HTTP/HTTPS requests to ECS Tasks and EKS Pods across multiple Availability Zones.
- **Amazon EKS integrates with AWS IAM via IRSA:** IAM Roles for Service Accounts allows individual Kubernetes Pods to assume specific, granular IAM permissions.
- **Amazon ECS uses Task Roles for application permissions:** ECS assigns IAM Task Roles directly to running containers to authorize access to AWS services like DynamoDB and S3.
- **Amazon VPC isolates container compute networks:** All EC2 worker nodes, ECS tasks, and Fargate instances run inside isolated VPC subnets secured by Security Groups.

8. Key Takeaways, Entities & Interview Questions

8.1 Key Takeaways

- Amazon EC2 provides virtual servers, while Amazon ECS and Amazon EKS provide managed container orchestration platforms.

- AWS Fargate removes server management completely by providing a serverless, pay-as-you-go compute engine for containers.
- Amazon ECR acts as the central container image repository for all AWS compute services.
- Security is enforced at the individual container level using IAM Roles for Service Accounts (IRSA) in EKS and Task Roles in ECS.
- High availability is achieved by distributing container workloads across multiple AWS Availability Zones behind an Application Load Balancer.

8.2 Important Entities & Classification

- **AWS Compute Services:** Amazon EC2, AWS Fargate, AWS Auto Scaling, AWS Nitro System.
- **Container Orchestration:** Amazon ECS (Elastic Container Service), Amazon EKS (Elastic Kubernetes Service).
- **Container Repositories & Artifacts:** Amazon ECR (Elastic Container Registry), Task Definition, Container Image, Pod.
- **Networking & Routing:** AWS VPC (Virtual Private Cloud), Subnets, Application Load Balancer (ALB), Target Groups, Security Groups.
- **Security & Identity:** AWS IAM (Identity and Access Management), IAM Role, ServiceAccount, IRSA, Task Role, Task Execution Role.
- **Storage & Databases:** Amazon EBS, Amazon S3, Amazon RDS, Amazon DynamoDB.

8.3 Common Technical & Interview Questions

1. When should an organization choose Amazon ECS over Amazon EKS?

Answer: Amazon ECS is ideal for teams deeply invested in the AWS ecosystem who want a simple, powerful container orchestrator with minimal operational overhead and native AWS integrations. Amazon EKS is preferred when an organization already uses Kubernetes, requires open-source tooling portability across multiple cloud providers, or needs advanced Kubernetes custom resource definitions (CRDs).

2. What is AWS Fargate, and how does it change how containers are run?

Answer: AWS Fargate is a serverless compute engine for containers. Instead of provisioning, managing, and patching EC2 virtual machines, you specify the CPU and memory your container requires, and AWS runs it in an isolated compute environment, billing you only for the exact seconds the container executes.

3. How does IAM Roles for Service Accounts (IRSA) improve container security in Amazon EKS?

Answer: Without IRSA, all containers on an EC2 worker node inherit the permissions of the underlying node's IAM role. IRSA binds IAM roles directly to specific Kubernetes ServiceAccounts, ensuring that each microservice receives only the exact permissions it needs, adhering to the principle of least privilege.

4. What is the difference between an ECS Task Execution Role and an ECS Task Role?

Answer: The Task Execution Role is used by the ECS infrastructure agent to pull images from Amazon ECR and send logs to CloudWatch. The Task Role is used by the application code running inside the container to interact with AWS services like DynamoDB, S3, or SQS.

5. How does an AWS Application Load Balancer distribute traffic to healthy containers?

Answer: The ALB registers container IP addresses and ports as targets inside a Target Group. It periodically sends HTTP health check requests to a configured path (e.g., /healthz). If a container fails consecutive health checks, the ALB stops sending traffic to it and routes requests to remaining healthy containers.